OpenFLIGHT

Technical Manual: Aerodynamic Model Generator and 6-DoF Simulator

Generated from the code base at
`C:\___DEV___\OpenFLIGHT\2026_04_13_OpenFLIGHT_`

April 14, 2026

Abstract

OpenFLIGHT is a browser-driven, Julia-based flight simulator for aerobatic and general-aviation aircraft. The toolchain consists of two main components: (i) the *Aircraft Model Creator*, which converts a geometric aircraft definition into a dense, physics-based aerodynamic database covering the full ±180° flight envelope; and (ii) the *flight simulator proper*, a time-stepped 6-degree-of-freedom (6-DoF) rigid-body solver that consumes that database to integrate the aircraft trajectory in real time. This manual documents, in depth, the numerical and aerodynamic formulations used by both stages, how data flows between modules, and the rationale for each modelling choice. It is aimed at readers familiar with classical flight dynamics (Etkin, Roskam, Stevens–Lewis) who want to understand how the specific code at hand translates textbook theory into a working simulator.

Contents

1 System architecture **2**
1.1 Coordinate frames and conventions . . . 2
1.2 Schema versions . . . 3

2 Aircraft definition file **3**
2.1 NACA airfoil parsing . . . 4
2.2 Planform geometry . . . 4

3 Aerodynamic Model Generator **4**
3.1 Pipeline overview . . . 4
3.2 VLM backend . . . 5
3.3 JAVL backend . . . 5
3.4 DATCOM backend . . . 6
3.5 Schema v3.0 component split . . . 7
3.6 Tail aerodynamic center and moment transfer . . . 8
3.7 Full-envelope extension (±180°) . . . 8
3.8 Data sources and selection criteria . . . 9
3.8.1 What each backend delivers . . . 9
3.8.2 What they don't deliver well . . . 10
3.8.3 Per-coefficient-class priority . . . 10
3.8.4 What the v3.0 split blocks draw from . . . 11
3.8.5 Illustration: one PC-21 coefficient, tracked through the pipeline . . . 11
3.8.6 What happens when a backend is missing . . . 12
3.8.7 Validation and quality report . . . 12
3.9 YAML output . . . 12

4 Flight simulator — coefficient assembly **13**
4.1 Database loading and aliasing . . . 13
4.2 Tensor interpolation . . . 13
4.3 Flight conditions and anemometry . . . 13
4.4 Interference and local tail angles . . . 14
4.5 Per-surface force and moment assembly . . . 14
4.6 Total whole-aircraft coefficient . . . 14
4.7 Dynamic stall blend on wing+body . . . 15
4.8 Worked example — PC21 cruise lookup . . . 15

5 Flight simulator — full moment sum **15**
5.1 Stiffness dispatch . . . 16
5.2 Post-stall damping floor . . . 16

6 6-DoF equations of motion **17**
6.1 State vector . . . 17
6.2 Newton–Euler equations . . . 17
6.3 Inertia tensor . . . 17
6.4 Quaternion kinematics . . . 17
6.5 Runge–Kutta 4 integrator . . . 18

7 Anemometry, atmosphere, controls **18**
7.1 ISA-76 atmosphere . . . 18
7.2 Actuator and engine dynamics . . . 18
7.3 Control attainment . . . 18

8 Visualization and inspection **19**
8.1 Aero model viewer (`aero_model_viewer.html`) . . . 19
8.2 Model Creator results panel . . . 19

9 Nomenclature **19**

A Program flow **19**
A.1 Model Creator startup . . . 20
A.2 Flight simulator startup . . . 20
A.3 Per-frame integration flow . . . 20

B Function reference **21**
B.1 Model Creator — AeroModel module . . . 22
B.2 Flight simulator . . . 22
B.3 Source file map (at-a-glance) . . . 23

1 System architecture

OpenFLIGHT is split into two Julia applications that communicate with a browser front-end over WebSockets:

- **Model Creator** — `RunModelCreator.jl` launches the module `AeroModel`. It ingests the geometric aircraft definition (`*.ac_data.yaml`), runs one or more aerodynamic backends (VLM, JAVL, DATCOM), merges the results into a unified schema v3.0 database, and writes that database as `*.tabular.aero_prop.yaml` (for full table mode) and `*.linearized.aero_prop.yaml` (for a scalar-derivative fallback mode).
- **Flight simulator** — `OpenFLIGHT.jl` loads the YAML database, runs a real-time RK4 integrator on the rigid-body equations of motion, and streams the resulting state to the browser for visualization and flight control.

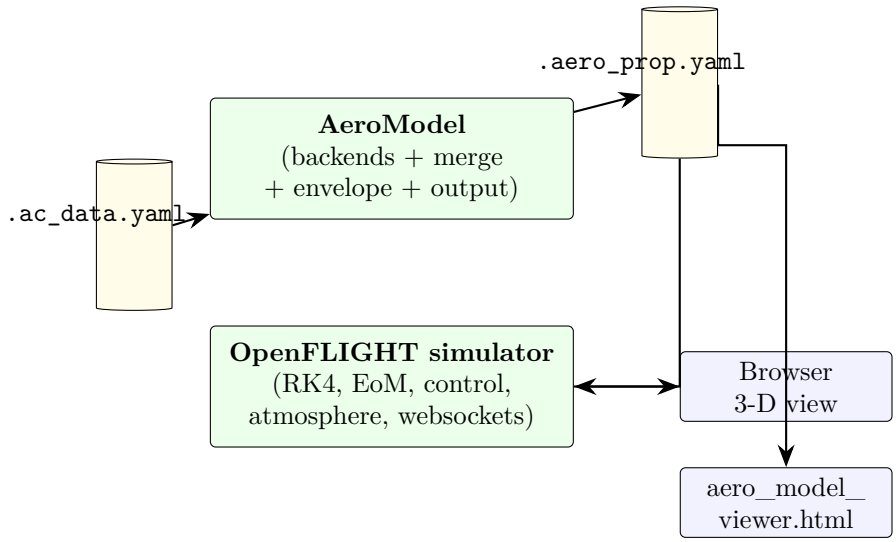


Figure 1: End-to-end data flow. The aerodynamic database generated by `AeroModel` is consumed by the simulator at load time and, in parallel, by the `aero_model_viewer` for offline inspection.

The two entry points `OpenFLIGHT.jl` and `RunModelCreator.jl` resolve required Julia packages on first run, find a free WebSocket port, inject that port into the JavaScript front-end, open the browser, and then block in a server loop waiting for client events. The browser is the only user interface — there is no separate desktop application.

1.1 Coordinate frames and conventions

The simulator uses two right-handed reference frames consistently throughout:

Aerodynamic body axes (x forward, y right, z down)
This is the convention assumed by every YAML database entry and by all aerodynamic code (`0.2.1`–`0.2.5`). Roll rate p is positive right-wing-down about x; pitch rate q is positive nose-up about y; yaw rate r is positive nose-right about z. The freestream velocity in body axes is $\boldsymbol{V} = (V\cos\alpha\cos\beta,\ V\sin\beta,\ V\sin\alpha\cos\beta)$.

Simulator global axes (x east, y up, z north)
This is the frame used by the physics integrator, by the quaternion orientation state, and by the browser renderer.

A quaternion $\boldsymbol{q} = (q_1, q_2, q_3, q_0)^\top$ (vector-first, scalar last) rotates vectors between the two frames. Any per-axis sign flip needed to reconcile the two conventions (for instance the $\dot{\beta} \approx -r$ identity, or the overall negative sign on the rolling-moment dihedral effect) is documented with an explanatory comment at the call site and preserved across the component-assembly rewiring described in §4.

1.2 Schema versions

OpenFLIGHT has used three aerodynamic-database schemas so far. Understanding the differences matters because the simulator dispatches at load time on the presence of the v3.0 blocks:

- **v2.x** — whole-aircraft coefficients only; the tail contribution is implicit in C_{m_α}, C_{n_β}, etc.
- **v3.0** (current) — adds three sibling blocks `wing_body`, `tail` (a list of per-surface entries), and `interference` ($\varepsilon, \sigma, \eta_h, \eta_v$ tables). The whole-aircraft `static_coefficients` block is still emitted alongside as a plotting overlay.
- **v3.0+** — adds the lag derivatives $C_{m_{\dot{\alpha}}}$ and $C_{n_{\dot{\beta}}}$ to the `dynamic_derivatives` block for short-period and Dutch-roll damping augmentation.

The simulator prefers the v3.0 assembler path when `wb_CL` (or any wing+body coefficient) is present in the database, and falls back to the legacy whole-aircraft path otherwise — so pre-v3 aircraft keep flying unchanged.

2 Aircraft definition file

An aircraft is defined by a single YAML file with extension `.ac_data.yaml`. The parser is in `AeroModel/src/AeroModel/input.jl`. The file is intentionally *aerodynamic-hint-free*: it contains mass, inertia, geometry, engines, and analysis options, but no directly supplied aerodynamic coefficients. All nonlinear characteristics are derived from geometry by the backends.

```yaml
general:
  aircraft_reference_surface_m2: 16.3
  aircraft_reference_mean_aerodynamic_chord_m: 1.79
  aircraft_reference_span_m: 9.11
  aircraft_CoG_coords_xyz_m: [2.95, 0, 0]
  mass_kg: 1600
  inertia:
    principal_moments_kgm2: { Ixx_p: 2800, Iyy_p: 5200, Izz_p: 7400 }
    principal_axes_rotation_deg: { roll: 0, pitch: -2, yaw: 0 }

lifting_surfaces:
  - name: Wing # role="wing" -> wing+body bucket
    root_LE: [2.4, 0, -0.1]
    AR: 5.1, TR: 0.45, dihedral_DEG: 5, sweep_quarter_chord_DEG: 6
    surface_area_m2: 16.3, mean_aerodynamic_chord_m: 1.79
    incidence_DEG: 2, twist_tip_DEG: -2
    airfoil: { type: NACA, root: "23015", tip: "23012" }
    control_surfaces: [ aileron { eta:0.62-0.95, cf:0.25 },
                        flap { eta:0.15-0.58, cf:0.30 } ]
  - name: HTail # role="horizontal_stabilizer" -> tail_h
  - name: VTail # role="vertical_stabilizer" -> tail_v

fuselages: [{ name: Fuselage, diameter, length, nose_position }]
engines: [{ id: PT6A, position_m, orientation_deg, max_thrust_n, ... }]
configurations: [{id: clean, flap_deg: 0, gear: up}, {id: landing, ...}]
analysis: alpha_range_DEG, beta_range_DEG, mach_values, backends: [javl, datcom]
```

The loader classifies each lifting surface by role (`component_split.jl::classify_role`) into one of three buckets:

$$\text{role} \in \{\text{wing_body},\ \text{tail_h},\ \text{tail_v}\}.$$

Anything matching *wing*, *canard*, or containing a role string not recognised as a stabilizer falls into `wing_body`. Recognised horizontal-tail strings include `horizontal_stabilizer`, `canard`, `h_tail`, `stab`; recognised vertical-tail strings include `vertical_stabilizer`, `v_tail`, `fin`, `ventral_fin`.

2.1 NACA airfoil parsing

The airfoil designator is translated into three geometric numbers (thickness ratio t/c, maximum camber m, camber position p) by `parse_naca_geometry`. Four-digit codes (e.g. 2412) are decoded as $m = M/100$, $p = P/10$, $t = TT/100$; five-digit codes (e.g. 23015) use the ideal lift coefficient relation $m \approx C_{L,i}/(4\pi)$ with $C_{L,i} = 0.15L$. The leading-edge radius is taken from the classical NACA four-digit formula $r_{LE}/c \approx 1.1019(t/c)^2$.

These three numbers feed the DATCOM stall estimator (maximum lift coefficient, leading-edge vs. trailing-edge stall type), the VLM camber line (so that VLM sees a cambered surface rather than a flat plate), and the induced-drag factor (Oswald efficiency from aspect ratio).

2.2 Planform geometry

The function `wing_planform(surf)` converts the user-supplied (area, aspect ratio, taper ratio, quarter-chord sweep, dihedral) into the quantities every backend needs:

$$b = \sqrt{S \cdot AR}, \qquad c_r = \frac{2S}{b(1+\lambda)}, \qquad c_t = \lambda c_r,$$

$$\bar{c} = \tfrac{2}{3}\, c_r\, \frac{1+\lambda+\lambda^2}{1+\lambda}, \qquad \Lambda_{LE} = \arctan\!\big(\tan\Lambda_{c/4} + \tfrac{c_r(1-\lambda)}{2(b/2)}\big),$$

and the MAC station $y_{\bar{c}} = (1+2\lambda)/(3(1+\lambda)) \cdot b/2$, needed for locating the aerodynamic center (see §3.6).

3 Aerodynamic Model Generator

3.1 Pipeline overview

The top-level function `run_analysis` in `AeroModel.jl` carries out the following steps:

1. **Sanitise** the input JSON, stripping derived aerodynamic hints (`stall_parameters`, `dynamic_stall`, `tail_properties`, `Oswald_factor`, `sideslip_drag_K`, `scale_tail_forces`). These are either UI artefacts or quantities computed downstream from geometry.
2. **Parse** into a typed `AircraftInput` struct (`parse_aircraft_input`).
3. **Run backends** in the user-specified order: VLM, JAVL, DATCOM. Each produces an independent Dict of results. Backend failures do not abort the pipeline — the merge stage falls back to the next available source.
4. **Extend to** $\pm 180°$ via `extend_to_full_envelope`. This stage attaches itself to the VLM result dict even if VLM didn't run (in that case it fabricates the linear derivatives analytically from geometry).
5. **Merge** the three sources into a unified schema-v3.0 `model` Dict (`merge_results`).
6. **Validate** the model (`validate_aero_model`) and attach a quality report.

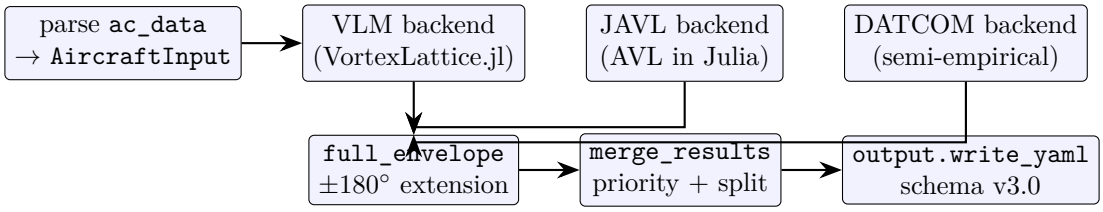


Figure 2: Aerodynamic model generator pipeline. Each backend operates independently; the full-envelope stage extends the small-angle linear solutions to ±180°; `merge_results` picks a priority source per coefficient and adds the schema-v3.0 component split; the output writer emits the unified YAML.

3.2 VLM backend

The VLM backend (`vlm_backend.jl`) is a thin adapter around `VortexLattice.jl`, a steady vortex-lattice solver. For every requested (α, β, M) inside the potential-flow validity window ($|\alpha| < 25°$, $|\beta| < 25°$) it solves the linear system

$$\mathbf{A}\,\boldsymbol{\Gamma} \;=\; \boldsymbol{b}(\alpha, \beta), \qquad \mathbf{A}_{ij} = \frac{\boldsymbol{V}_{j\to i} \cdot \hat{\boldsymbol{n}}_i}{V_\infty}, \tag{1}$$

where $\mathbf{A}$ is the aerodynamic influence coefficient matrix (geometry-only, built once per mesh), $\boldsymbol{\Gamma}$ is the vector of panel circulation strengths to solve for, $\hat{\boldsymbol{n}}_i$ is the outward normal on panel i, and $\boldsymbol{b}$ is the freestream normal component. Once $\boldsymbol{\Gamma}$ is known, the Kutta–Joukowski law gives the force per panel:

$$\boldsymbol{F}_i \;=\; \rho\, \boldsymbol{V}_i \times \boldsymbol{\Gamma}_i\, \ell_i, \tag{2}$$

which is summed over all panels to produce the whole-aircraft non-dimensional forces C_F and moments C_M about the reference point. The module invokes `VortexLattice.body_forces` to get these sums directly.

Meshing strategy. In `build_vlm_geometry` each lifting surface is discretised with a panel count proportional to its span-to-chord ratio ($n_s = \max(7,\ \mathrm{AR} \cdot n_c)$, $n_c = 5$ minimum). Symmetric surfaces are always modelled with explicit mirroring — the alternative VortexLattice symmetry boundary condition forces $C_Y = 0$ at all sideslip angles, which is a catastrophic loss for lateral dynamics. Each fuselage is wrapped as an octagon (8 flat-panel sides along the fuselage length), which is crude but adds non-zero slender-body side-force and is cheap.

Per-surface extraction. The v3.0 pipeline needs forces and moments *per lifting surface*, not only the whole-aircraft sum. `extract_per_surface_forces` walks the panel array and accumulates the Kutta–Joukowski force of each surface's panels separately, returning a vector of (CF_i, CM_i) tuples indexed by surface. The moments are initially computed about the reference point (aircraft CoG); the tail-AC transfer happens later in the assembler.

Stability derivatives. VortexLattice provides `stability_derivatives` which returns the Jacobians of (C_D, C_Y, C_L) and (C_l, C_m, C_n) with respect to $(\hat{\alpha}, \hat{\beta}, \hat{p}, \hat{q}, \hat{r})$ at the current operating point. Of these, the simulator uses: $\partial C_l/\partial \hat{p}$ (roll damping), $\partial C_m/\partial \hat{q}$ (pitch damping), $\partial C_n/\partial \hat{r}$ (yaw damping), $\partial C_L/\partial \hat{q}$ (tail lift from pitch rate), $\partial C_Y/\partial \hat{p}$ (VTP side force from roll rate), $\partial C_Y/\partial \hat{r}$ (VTP side force from yaw rate).

3.3 JAVL backend

JAVL (`javl_backend.jl`) is a Julia port of Mark Drela's AVL. It converts the aircraft geometry to an AVL input string (the familiar `.avl` format) and invokes a Julia AIC solver identical in

spirit to (1). Its main advantage over raw VortexLattice is that it exposes section-level data (`sol.strip_cl`) used by the v3.0 pipeline to aggregate per-surface lift. Its main limitation is that the current integration runs only at $\beta = 0$, so JAVL contributes only to the longitudinal sector of the database.

3.4 DATCOM backend

The DATCOM backend (`datcom_backend.jl`) is a complete re-implementation of USAF DATCOM 1978's semi-empirical handbook methods, in Julia. It runs on a (mach, α) grid and produces whole-aircraft coefficients via explicit formulas. Its value is that it does not depend on a potential-flow solve — it is fast, it gives full Mach coverage, and its dynamic derivatives are validated against flight test data. The simulator consumes DATCOM primarily for the dynamic and cross-damping derivatives.

Lift curve — Helmbold with compressibility.

$$C_{L_\alpha} \;=\; \frac{C_{L_\alpha}^{2D} \cdot AR}{2 + \sqrt{4 + (AR\,\beta_{PG}/\kappa)^2\,(1 + \tan^2\Lambda/\beta_{PG}^2)}}, \qquad \beta_{PG} = \sqrt{|1 - M^2|},$$

where $C_{L_\alpha}^{2D} = 2\pi$ is the thin-airfoil value, κ is a section efficiency (unity in this code), and $\Lambda = \Lambda_{c/4}$ is the quarter-chord sweep. This is applied independently to wing, horizontal tail, and vertical tail, yielding $C_{L_{\alpha,\text{wing}}}$, $C_{L_{\alpha,h}}$, $C_{L_{\alpha,v}}$.

Tail contribution to lift. The tail-on lift-curve slope is assembled as

$$C_{L_\alpha}^{\text{total}} \;=\; C_{L_{\alpha,\text{wing}}} \;+\; C_{L_{\alpha,h}} \cdot \frac{S_h}{S_{\text{ref}}}\,(1 - \tfrac{d\varepsilon}{d\alpha})\,\eta_t, \qquad \frac{d\varepsilon}{d\alpha} \;\approx\; \frac{2\,C_{L_{\alpha,\text{wing}}}}{\pi AR}, \tag{3}$$

and η_t is the tail dynamic-pressure ratio derived from the vertical offset of the tail relative to the wing wake: $\eta_t = 1 - 0.12\exp[-(z_h/\Delta_{\text{wake}})^2]$, with $\Delta_{\text{wake}} = 0.12\,c$.

Stall model. The 3-D maximum lift coefficient is obtained from a 2-D section value $c_{\ell,\max}(t/c,\ \text{camber},\ \text{Re})$ through a knockdown for aspect ratio, taper ratio, and sweep:

$$C_{L,\max} \;=\; c_{\ell,\max} \cdot \underbrace{(0.80 + 0.20\lambda) \cdot \text{clamp}(0.95 - 0.01\max(AR - 6, 0), 0.85, 0.98) \cdot \sqrt{\cos\Lambda}}_{k_{3D}\in[0.70,1.0]}. \tag{4}$$

The post-stall decay is an exponential blend with a flat-plate limit, parametrised by a stall type (leading-edge vs. trailing-edge) inferred from t/c (thin $\to$ LE, thick $\to$ TE).

Drag build-up.

$$C_D \;=\; C_{D_0}(\text{Re},\, t/c) \;+\; \frac{C_L^2}{\pi AR\, e} \;+\; C_{D_{\text{wave}}}(M) \;+\; C_{D_{\text{cross}}}(\beta),$$

with skin-friction $C_f = 0.074/\text{Re}_x^{0.2}$ (Schoenherr form), profile-to-friction form-factor correction for wings, bodies, and tails, Oswald efficiency estimated from aspect ratio, and a Prandtl–Glauert wave-drag onset for $M \gtrsim 0.7$.

Dynamic derivatives — the key closed-form results. The simulator consumes the following five damping/cross derivatives. Each is exposed by `datcom_calculate_point` with the closed form:

$$C_{L_q} = \quad 2\, C_{L_{\alpha,h}}\, \eta_t\, \frac{S_h}{S_{\text{ref}}}\, \frac{\ell_t}{c_{\text{ref}}}\, \cos\alpha\, \eta_{\text{tail}}(\alpha), \tag{5}$$

$$C_{m_q} = -2\, C_{L_{\alpha,h}}\, \eta_t\, V_H\, \frac{\ell_t}{c_{\text{ref}}}\, \cos\alpha\, \eta_{\text{tail}}(\alpha) \;=\; -2\, C_{L_{\alpha,h}}\, \eta_t\, \frac{S_h}{S_{\text{ref}}} \left(\frac{\ell_t}{c_{\text{ref}}}\right)^2 \cos\alpha\, \eta_{\text{tail}}(\alpha), \tag{6}$$

$$C_{n_r} = -2\, C_{L_{\alpha,v}}\, \eta_v\, \frac{S_v}{S_{\text{ref}}} \left(\frac{\ell_v}{b_{\text{ref}}}\right)^2 \cos\alpha\, \eta_{\text{vtp}}(\alpha), \tag{7}$$

$$C_{l_p} = -\frac{C_{L_{\alpha,w}}}{12}\, \frac{1+3\lambda}{1+\lambda}\, \cos\Lambda\, \cos\alpha\, \eta_{\text{wing}}(\alpha), \tag{8}$$

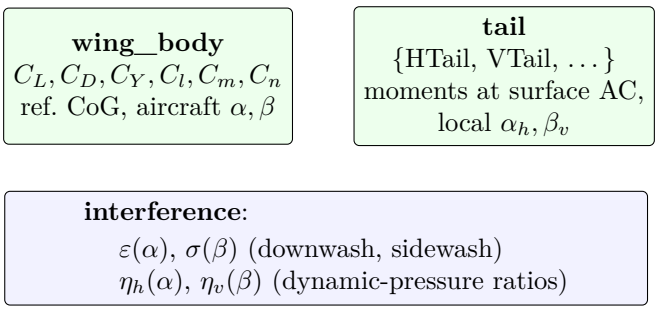
$$C_{m_{\dot\alpha}} = C_{m_q}\, \frac{d\varepsilon}{d\alpha}, \qquad C_{n_{\dot\beta}} = C_{n_r}\, \frac{d\sigma}{d\beta}, \qquad \frac{d\sigma}{d\beta} \approx 0.1. \tag{9}$$

The $(\ell_t/c_{\text{ref}})^2$ factor in (6) — and its sibling $(\ell_v/b_{\text{ref}})^2$ in (7) — deserve a word of explanation, because they encode one of the single most important physics facts of a conventional aircraft. The tail sees a local-α perturbation $\Delta\alpha_h = q\,\ell_t/V$ when the aircraft pitches at rate q (its leading edge moves down at $q\,\ell_t$ relative to the CoG). That perturbation produces a lift $\Delta L = C_{L_{\alpha,h}} \Delta\alpha_h\, q_t S_h$. The lift acts at ℓ_t behind the CoG, giving a pitching moment $\Delta M = \Delta L \cdot (-\ell_t)$. Non-dimensionalising by $q_\infty S\, c$ and identifying $\hat q = qc/(2V)$, both ℓ_t factors appear — one from the local-angle perturbation and one from the moment arm — giving the famous ℓ_t^2 scaling. Yaw damping is the exact analogue with ℓ_v and b.

The lag derivatives $C_{m_{\dot\alpha}}$ and $C_{n_{\dot\beta}}$ in (9) capture the time delay between a change in free-stream α, β and the arrival of the corresponding downwash/sidewash at the tail. For a rigid body in quasi-static flight, $\dot\alpha \approx q$ and $\dot\beta \approx -r$, so these derivatives effectively augment C_{m_q} and C_{n_r} by the factors $d\varepsilon/d\alpha$ and $d\sigma/d\beta$ — typically +30% to pitch damping for a conventional tail.

3.5 Schema v3.0 component split

Rather than emit a single whole-aircraft coefficient set, the generator stores aerodynamics as three independent blocks plus one interference block:



wing_body. Tabulated in aircraft-level angles (α, β), in the aircraft CoG frame, non-dimensional to $(S_{\text{ref}}, c_{\text{ref}}, b_{\text{ref}})$. Contains $C_L, C_D, C_Y, C_l, C_m, C_n$ as 4-D tensors indexed by $(\text{config}, \text{mach}, \alpha, \beta)$.

tail. A list of per-surface entries, one per tail surface (`HTail`, `VTail`, canard, etc.). Each entry carries:

- `name, role, component` — metadata.
- `arm_m` = $\boldsymbol{r}_{AC} - \boldsymbol{r}_{CoG}$ (metres, body axes) — used by the simulator for the $\boldsymbol{r} \times \boldsymbol{F}$ moment transfer.
- `ac_xyz_m` — the tail's aerodynamic center in body axes.

- The six coefficient tensors $C_L, C_D, C_Y, C_{l,AC}, C_{m,AC}, C_{n,AC}$, tabulated in *local* angles (α_h, β_v). Moments are taken about the tail's own AC, *not* about the aircraft CoG. Non-dimensional to aircraft S_{ref}.

interference. Four 1-D tables: $\varepsilon(\alpha)$, $\sigma(\beta)$, $\eta_h(\alpha)$, $\eta_v(\beta)$, each also indexed by (config, mach). The simulator evaluates these at every step to derive the local tail angles and dynamic-pressure ratios.

The simulator reassembles the global coefficients from these components at every integration step — see §4.

3.6 Tail aerodynamic center and moment transfer

The location of a surface's aerodynamic center is computed by `component_split.jl::surface_aerodynamic_cente` For a non-vertical symmetric pair, the AC sits on the centreline (by symmetry), at longitudinal station

$$x_{AC} \;=\; x_{LE,\text{root}} \;+\; y_{\bar{c}} \tan\Lambda_{LE} \;+\; \tfrac{1}{4}\bar{c}, \tag{10}$$

and at the vertical station derived from the dihedral-projected MAC-station elevation. For a vertical surface the "span" is interpreted along the $-z$ axis (fin tip higher), so $y_{AC} = y_{LE}$ and $z_{AC} = z_{LE} - y_{\bar{c}}$.

The transfer from the tail AC to the aircraft CoG is the standard rigid-body relation

$$\boldsymbol{M}_{\text{CoG}} \;=\; \boldsymbol{M}_{\text{AC}} \;+\; (\boldsymbol{r}_{\text{AC}} - \boldsymbol{r}_{\text{CoG}}) \times \boldsymbol{F}, \tag{11}$$

implemented by the helper `component_split.jl::translate_moment_coefficients`. That function takes care of the per-axis reference-length scaling automatically: dimensional force has units of $\bar{q}_\infty S_{\text{ref}}$, but the moment coefficients about the three body axes are non-dimensional to $\bar{q}_\infty S_{\text{ref}}\, b_{\text{ref}}$, $\bar{q}_\infty S_{\text{ref}}\, c_{\text{ref}}$, $\bar{q}_\infty S_{\text{ref}}\, b_{\text{ref}}$ respectively. The code therefore writes

$$\begin{aligned} \Delta C_l\big|_{\text{CoG}} &= C_{l,\text{AC}} + (r_y F_z - r_z F_y)/b_{\text{ref}}, \\ \Delta C_m\big|_{\text{CoG}} &= C_{m,\text{AC}} + (r_z F_x - r_x F_z)/c_{\text{ref}}, \\ \Delta C_n\big|_{\text{CoG}} &= C_{n,\text{AC}} + (r_x F_y - r_y F_x)/b_{\text{ref}}. \end{aligned} \tag{12}$$

3.7 Full-envelope extension ($\pm 180°$)

Potential-flow methods are only valid in the small-angle regime. `full_envelope.jl` extends the VLM (or analytic) linear solution to the full $\pm 180°$ range using DATCOM-style phenomenological models.

Longitudinal. For $|\alpha| < \alpha_{\text{stall}}$:

$$C_L(\alpha) \;=\; C_{L_0} + C_{L_\alpha}\, \alpha^*,$$

with α^* the soft-saturated α (smooth-step form of a clamp). Beyond stall:

$$C_L(\alpha) \;=\; \underbrace{\text{sgn}(\alpha)\, C_{L,\max}\, e^{-\Delta/\tau}}_{\text{peak + exponential fade}} \;+\; \underbrace{C_{D_{90}} \sin\alpha \cos\alpha\, (1 - e^{-\Delta/\tau})}_{\text{Hoerner flat-plate blend}},$$

with $\Delta = |\alpha| - \alpha_{\text{stall}}$ and time constant $\tau = 12/18/25°$ for leading-edge/combined/trailing-edge stall respectively. The flat-plate drag at $\alpha = 90°$ is $C_{D_{90}} \approx 1.98$ for a finite wing (Hoerner), reduced for small aspect ratio.

Drag is a simple parabolic in α with a cross-flow term in β:

$$C_D(\alpha, \beta) \;=\; C_{D_0} + (C_{D_{90}} - C_{D_0}) \sin^2\alpha + K_\beta\, C_Y^2. \tag{13}$$

Pitching moment uses the same linear-through-stall construction with C_{m_α} as slope and C_{m_0} as offset, and a decay toward zero beyond the stall angle.

Lateral-directional. The lateral coefficients are *not* linear in β over the full envelope. The fuselage contributes a Munk slender-body yawing moment $\propto \sin 2\beta$ (maximal at $\beta = 45°$) and a cross-flow term $\propto \sin\beta|\sin\beta|$, both destabilising. The vertical tail contributes a linear-through-stall side force that stalls at $|\beta_v| \gtrsim 20°$. Adding them:

$$C_n(\alpha,\beta) \;=\; \underbrace{C_{n_{\beta,\mathrm{vtp}}}\,\beta^*}_{\text{VTP stabilising}} \;+\; \underbrace{C_{n,\mathrm{body}}\,\sin 2\beta + C_{D_{90,\mathrm{lat}}}\,\sin\beta|\sin\beta|}_{\text{fuselage destabilising}}, \tag{14}$$

$$C_l(\alpha,\beta) \;= C_{l_{\beta,\mathrm{vtp}}}\,\beta^* \;+\; C_{l,\mathrm{body}}\,\sin 2\beta. \tag{15}$$

The VTP/body decomposition is computed inside `full_envelope.jl` by evaluating a geometry-based estimate of the body contribution and subtracting it from the total to leave a VTP-alone slope. This decomposition is a transient numerical trick — the final `wing_body.Cl` table still carries the sum. But it is critical for getting the right $\pm 180°$ behaviour: a linear extrapolation of $C_{n_\beta}\beta$ alone would give impossibly large restoring moments at $\beta = 90°$.

Split-block construction. The function `extend_split_blocks` uses the interference tables (from VLM or from an analytic placeholder) to compute, in local tail angles, the tail contribution to each whole-aircraft coefficient on the full grid:

$$C_{X,\mathrm{tail}}(\alpha,\beta) \;=\; \sum_{i\in\mathrm{tails}} \eta_i(\alpha \text{ or } \beta)\, C_{X,i}(\alpha - \varepsilon(\alpha),\ \beta - \sigma(\beta))\,\frac{S_i}{S_{\mathrm{ref}}}.$$

The wing+body block is then obtained by subtraction from the whole-aircraft envelope:

$$C_{X,\mathrm{wb}} \;=\; C_{X,\mathrm{env}} - C_{X,\mathrm{tail}}.$$

This is the only place where wb is constructed analytically; VLM provides it directly from its per-role aggregation in the narrow-angle range.

3.8 Data sources and selection criteria

The generator has three potential aerodynamic data sources, plus the envelope extender that synthesises data analytically when nothing else is available. No single source covers every coefficient well — each has well-understood failure modes — so the merge stage implements an explicit per-coefficient-class priority. This section explains what each source produces, what it *doesn't*, and the physical reasons behind each selection.

3.8.1 What each backend delivers

Source	Domain of validity	What it produces
VLM (`vlm_backend.jl`)	$\lvert\alpha\rvert, \lvert\beta\rvert \le 25°$; subsonic only; linear potential flow	Full (α,β) mesh of $C_L, C_D, C_Y, C_l, C_m, C_n$; stability derivatives at $\beta = 0$; per-surface forces for v3 aggregation; VLM panel geometry for 3-D rendering
JAVL (`javl_backend.jl`)	$\lvert\alpha\rvert \le 25°$, $\beta = 0$ only; subsonic; linear potential flow	$C_L, C_D, C_Y, C_l, C_m, C_n$ along a single $\beta = 0$ cut; control-surface derivatives from strip theory; per-surface CL from strip integration
DATCOM (`datcom_backend.jl`)	All Mach up to low-supersonic; pre-stall linear range in α; β *implicit only* through body crossflow	Whole-aircraft C_L, C_D, C_m over (mach, α) grid; closed-form dynamic derivatives; closed-form lag derivatives $C_{m_{\dot\alpha}}, C_{n_{\dot\beta}}$
Envelope (`full_envelope.jl`)	Full $\pm 180°$ by construction; phenomenological, not first-principles	Whole-aircraft + split blocks on the full grid; interference tables (analytic fallback when VLM absent)

The three "real" backends are orthogonal in what they do well. VLM is a first-principles potential-flow solver — it will tell you about a tailless delta or an unusual span-loading better than a handbook would, because it actually integrates the Biot–Savart field over the real geometry. JAVL is a specialised variant of the same method with a better per-strip output, making it the natural choice for control effectiveness (each hinge flap can be localised cleanly). DATCOM is semi-empirical: handbook formulas with Reynolds corrections, calibrated against decades of flight test. Of the three, DATCOM is the only one that covers Mach properly and the only one whose dynamic derivatives have been blessed by flight data.

3.8.2 What they don't deliver well

Each backend has a specific failure mode that the merge policy is designed to route around:

VLM — fuselage crossflow contamination
The fuselage in the OpenFLIGHT VLM mesh is represented as eight flat octagonal facets for computational economy. Flat panels in a potential flow generate circulation, and that circulation produces unrealistic lateral forces and moments in sideslip. In practice VLM over-reports $C_{Y_\beta}, C_{l_\beta}, C_{n_\beta}$ by up to an order of magnitude, and the *signs* of rate derivatives (C_{n_p}, C_{l_r}) are often wrong. For anything involving fuselage lateral response, VLM is therefore not trusted.

VLM — stall and post-stall
Potential flow is by definition attached-flow. VLM's C_L grows linearly past α_{stall} and never breaks. The narrow validity window ($\pm 25°$) is enforced by `vlm_alpha_lo/hi` in the backend so we simply don't ask VLM for out-of-range values.

JAVL — no sideslip sweep
JAVL runs at $\beta = 0$ only (`betas_deg = [0.0]`). It therefore contributes nothing to the lateral sector. It is used for the longitudinal static coefficients and control effectiveness only.

JAVL — no fuselage solve
The bodies section of the AVL geometry is not wired (`setup_body!` not called). Fuselage contribution to lift and pitching moment is missing from JAVL.

DATCOM — no explicit β dependence
DATCOM calculates whole-aircraft C_L, C_D, C_m at a single (α, M) point. Sideslip effects enter only through the dynamic derivatives ($C_{Y_\beta}, C_{l_\beta}, \ldots$) and through the crossflow drag term. If DATCOM is the only available source, the resulting database has *no* β dependence in the static coefficients — a hard limitation for aerobatic flight.

DATCOM — linear pre-stall only
DATCOM delivers a single lift-curve slope and a single stall point. Anything in between is a linear extrapolation, and anything past it is the envelope extender's job.

Envelope — phenomenological, not predictive
The envelope equations in §3.7 are curve-fits designed to bridge the gap between the linear backends and the known $\alpha = 90°$ flat-plate asymptote. Accuracy at intermediate angles (e.g. $\alpha = 40°$ on a swept wing) is qualitative. It is the fallback of last resort.

3.8.3 Per-coefficient-class priority

With the capabilities and failure modes above, the merge policy is:

Block / coefficient	Priority (first wins)	Rationale
Static (C_L, C_D, C_m) in $\pm 25°$ range	VLM → DATCOM → JAVL	VLM carries the real (α, β) shape; DATCOM lacks β; JAVL is a narrow single-cut fallback.
Static lateral (C_Y, C_l, C_n) in $\pm 25°$ range	VLM (after envelope decomposition) → analytic (body + VTP)	VLM's raw lateral values are unreliable (fuselage cross-flow); the envelope VTP/body decomposition in §3.7 rescales them with a physical analytic model before use.
Outside $\pm 25°$ (all coefficients)	Envelope extender	No backend is valid outside small angles; envelope bridges to $\pm 180°$ analytically.
Dynamic derivatives $(C_{l_p}, C_{m_q}, C_{n_r}, \ldots)$	DATCOM → JAVL → VLM	DATCOM formulas are validated against flight test; VLM's rate derivatives are corrupted by fuselage panels (wrong signs, inflated magnitudes); JAVL is a subset of VLM's approach.
Lag derivatives $(C_{m_{\dot\alpha}}, C_{n_{\dot\beta}})$	DATCOM only	Neither potential-flow method models wake lag; DATCOM derives these as $C_{m_q}\, d\varepsilon/d\alpha$ etc. from classical theory.
Control effectiveness $(C_{l_{\delta_a}}, C_{m_{\delta_e}}, \ldots)$	JAVL → VLM → DATCOM	JAVL's strip theory localises each hinge cleanly; VLM's panel-based approach works but with more noise; DATCOM uses global flap τ which is coarse.
Local flow $(\varepsilon, \sigma, \eta_h, \eta_v)$	VLM-sampled (if wired) → analytic placeholder	Direct VLM induced-velocity sampling at tail panels is the gold standard; current code uses an analytic DATCOM-style fallback (Roskam formula for $d\varepsilon/d\alpha$) when VLM interference extraction isn't available.

The priority is encoded in `get_merged_coefficient` (static) and `get_merged_derivative` (dynamic) in `merge.jl`, both implementing the same "first non-null, ordered list" algorithm but with opposite orderings.

3.8.4 What the v3.0 split blocks draw from

The component-split blocks (`wing_body`, `tail`, `interference`) come through a slightly different path because they require per-surface data the backends produce in different degrees:

- `wing_body` block: constructed in `extend_split_blocks` as *whole-aircraft envelope minus analytic tail*. The whole-aircraft side uses the merge priority above; the tail side is reconstructed from geometry so the subtraction is self-consistent even when VLM supplied only the total.
- `tail` block: per-surface tables in local angles (α_h, β_v). VLM's `extract_per_surface_forces` provides per-role forces and moments at the tail AC in the $\pm 25°$ range; outside that range the envelope extender synthesises a per-surface envelope model using each surface's own aspect ratio, taper, and sweep. Moments at each tail's own AC remain a single value (the `Cm_at_AC` entry) because tail lift variation within the local angle range is what makes the contribution — the simulator's $r \times F$ handles the rest.
- `interference` block: prefers VLM-sampled induced velocities at tail panels when available; falls back to the analytic Roskam $d\varepsilon/d\alpha$ formula and a DATCOM η_h, η_v model. The current code ships with the analytic placeholder; the VLM-sampled path is marked as a future-phase enhancement.

3.8.5 Illustration: one PC-21 coefficient, tracked through the pipeline

Consider C_{m_q} for PC-21 at $\alpha = 0°, M = 0.2$, clean config. Which source delivers it?

1. DATCOM's `datcom_calculate_point` produces $C_{m_q} = -2\, C_{L_{\alpha,h}} \eta_t\, V_H\, \ell_t/c_{\text{ref}} \cdot \cos\alpha \cdot \eta_{\text{tail}}(\alpha)$. With $C_{L_{\alpha,h}} \approx 3.97/\text{rad}$, $V_H \approx 0.66$, $\ell_t/c_{\text{ref}} \approx 3.16$, $\cos 0 = 1$, $\eta_{\text{tail}} \approx 0.97$, this gives -14.85. Then the wing contribution adds ≈ -1.7, yielding the stored YAML value -16.52.
2. JAVL's dynamic derivatives contribute at $\beta = 0$; it produces a similar magnitude but without the compressibility factor, and with a slight discrepancy because of the missing body.

3. VLM's `stability_derivatives` also produces a C_{m_q}, but with wild numbers because the octagon fuselage contaminates the pitch-rate response.
4. `get_merged_derivative` walks the priority DATCOM → JAVL → VLM. DATCOM is present and non-null, so the DATCOM value (-16.52 after summing) wins. JAVL and VLM are discarded. This is what appears in `aerodynamics.dynamic_derivatives.Cm_q_hat` in the YAML.

The simulator then reads `Cm_q_hat`, the alias layer in `0.2.4` resolves the runtime call to `Cm_q`, and `0.2.2` uses it as the multiplier on $\hat{q}$.

3.8.6 What happens when a backend is missing

The pipeline degrades gracefully. If the user's analysis config sets `backends: [datcom]` only:

- Static coefficients come from DATCOM (no β dependence; the envelope extender interpolates DATCOM's single $\beta = 0$ cut across the full β range, using the analytic VTP+body decomposition for lateral coefficients).
- Dynamic derivatives come from DATCOM with no cross-check.
- Control effectiveness falls through to DATCOM's flap-τ method.
- Per-surface tail split is entirely analytic (envelope builds tail coefficients from geometry alone).
- Interference tables are the analytic DATCOM fallback.

The resulting database is still flight-capable but lacks the richness that VLM's (α, β) mesh provides.

Conversely if `backends: [vlm]` only:

- Static coefficients come from VLM directly in the $\pm 25°$ range; envelope extends beyond.
- Dynamic derivatives fall through to VLM, *which is precisely the problematic case.* The simulator will then show wrong-sign cross-damping unless the user provides overrides. The post-stall damping floor partially masks this but doesn't cure it.
- Lag derivatives default to zero (they are not part of VLM's output).

This is why the default `analysis.backends` in the examples ships as `[javl, datcom]` (or `[vlm, datcom]`): DATCOM is always recommended as a dynamic-derivative baseline.

3.8.7 Validation and quality report

After merge, `validate_aero_model` in `validation.jl` runs a battery of sanity checks: lift-curve slope sign and range, stall shape monotonicity, pitch-stability $C_{m_\alpha} < 0$, directional stability $C_{n_\beta} > 0$, C_Y antisymmetry in β, consistency of split-block sums with the whole-aircraft block, finiteness of every table entry. A per-check PASS/WARN/FAIL report is attached to the model under `model["quality"]["validation"]` and surfaced in the results panel.

The split blocks and interference are built by `build_wing_body_block`, `build_tail_block`, and `build_interference_block`, each reading from the highest-priority backend that supplies data in the required shape. If no backend supplied a surface's tail block, the full-envelope analytic reconstruction is used instead.

3.9 YAML output

Two writers emit the same YAML structure: the Julia-side `output.jl::write_yaml` and the JavaScript-side `customYamlDump` in `results-panel.js`. The reason there are two is that the Model Creator runs the computation in the Julia server but does the file save on the client side — the model Dict is shipped as JSON over the WebSocket and the JS formats it for download. Both

writers key off the same ordered key lists (`MODEL_KEY_ORDER`, `AERO_KEY_ORDER`, `YAML_LOOKUP_SET`) so the two outputs are textually identical.

The output schema is ordered as: `schema_version`, `meta`, `conventions`, `reference`, `limits`, `configurations`, `aerodynamics`:{`interpolation`, `static_coefficients`, `wing_body`, `tail`, `interference`, `dynamic_derivatives`, `control_effectiveness`, `control_drag_increments`, `local_flow`, `poststall`}, `per_surface_data`, `openflight_legacy`, `visual_geometry`, `propulsion`, `actuators`, `failures`, `quality`, `vlm_mesh`.

4 Flight simulator — coefficient assembly

At each integration step the simulator rebuilds the six whole-aircraft coefficients from the v3.0 blocks. The process is encapsulated in `0.2.5_assemble_from_components.jl`.

4.1 Database loading and aliasing

The YAML is loaded into an `AeroData` struct by `0.2.4_get_constants_and_interpolate_coefficients.jl`. The loader walks the `aerodynamics` dictionary and flattens each coefficient table into a top-level key with its own canonical axis order. The v3.0 extension re-exposes the new blocks under namespaced keys:

- `wing_body.CL` → `wb_CL`, and similarly for C_D, C_Y, C_l, C_m, C_n.
- `tail.surfaces[i].CL` → `tail_<Name>_CL`, `tail_<Name>_Cm_at_AC`, ..., with the local-angle axes `alpha_h_deg, beta_v_deg` canonicalised back to `alpha_deg, beta_deg` so the interpolation engine can use the same machinery.
- `interference.downwash_deg` → `interference_downwash_deg`, and similarly for the other three.

The aliasing layer additionally makes the suffix `_hat` and `_per_deg` optional, so runtime code fetching `Cm_q` transparently resolves to `Cm_q_hat` in the file.

4.2 Tensor interpolation

All coefficient tables are multilinear interpolants over the axes $(\text{config}, \text{mach}, \alpha, \beta)$. The lookup walker (`_interpolate_tensor_coefficient`) computes binary-weighted contributions over the 2^d neighbouring grid points, where d is the number of *numeric* axes (non-categorical). Out-of-range queries are clipped to the boundary and logged once per coefficient (the `clip_and_warn` policy in the `interpolation` block).

4.3 Flight conditions and anemometry

Before any aerodynamic lookup, the function `compute_instantaneous_flight_conditions` in `2.4` builds the NamedTuple consumed by `0.2.1-0.2.5`. It uses:

$$\boldsymbol{v}_\mathrm{b} = \mathcal{R}^{-1}(\boldsymbol{q})\,(\boldsymbol{v}_\mathrm{g} - \boldsymbol{v}_\mathrm{wind,g}), \tag{16}$$

$$V = \|\boldsymbol{v}_\mathrm{b}\|, \tag{17}$$

$$\alpha = \arctan\frac{v_{z,\mathrm{b}}}{v_{x,\mathrm{b}}}, \qquad \beta = \arcsin\frac{v_{y,\mathrm{b}}}{V}, \tag{18}$$

$$M = V/a(h), \qquad a(h) = \sqrt{\gamma R T(h)}, \tag{19}$$

with ρ, T from the ISA-76 atmosphere (`4.2_ISA76.jl`) and wind velocity from a configurable model in `4.3_anemometry.jl`.

α and β are then soft-saturated to the database boundary with a smooth-step knee so the interpolator never evaluates at the extrapolation edge:

$$\alpha_\mathrm{sat}(\alpha) = \alpha_\mathrm{min} + (\alpha_\mathrm{max} - \alpha_\mathrm{min}) \cdot \mathrm{smoothstep}\left(\frac{\alpha - \alpha_\mathrm{min} - \frac{1}{2}\alpha_\mathrm{knee}}{\alpha_\mathrm{max} - \alpha_\mathrm{min} - \alpha_\mathrm{knee}}\right). \tag{20}$$

4.4 Interference and local tail angles

In the assembler, four lookups retrieve the interference quantities at the current $(\alpha, \beta, \text{mach}, \text{config})$:

$$\varepsilon = \texttt{wb_fetch("interference_downwash_deg")}, \qquad \sigma = \texttt{wb_fetch("interference_sidewash_deg")},$$

$$\eta_h = \texttt{wb_fetch("interference_eta_h")}, \qquad \eta_v = \texttt{wb_fetch("interference_eta_v")}.$$

The local tail angles follow:

$$\alpha_h = \alpha - \varepsilon(\alpha), \qquad \beta_v = \beta - \sigma(\beta). \tag{21}$$

α_h is the angle the horizontal tail actually sees; β_v likewise for the vertical tail. These are the arguments at which the tail coefficient tables are looked up, so the tail sees the correct local flow.

4.5 Per-surface force and moment assembly

For each tail surface i, six lookups retrieve the surface's own contribution in its local frame:

$$C_{L,i} = C_{L,i}(\alpha_h, \beta_v), \qquad C_{D,i} = C_{D,i}(\alpha_h, \beta_v), \qquad C_{Y,i} = C_{Y,i}(\alpha_h, \beta_v),$$

$$C_{l,i,AC} = C_{l,i,AC}(\alpha_h, \beta_v), \qquad C_{m,i,AC} = C_{m,i,AC}(\alpha_h, \beta_v), \qquad C_{n,i,AC} = C_{n,i,AC}(\alpha_h, \beta_v).$$

A per-axis dynamic-pressure ratio is selected: $q_* = \eta_h$ for horizontal-tail surfaces, $q_* = \eta_v$ for vertical-tail surfaces. The scaled forces are

$$C_{L,i}^{\text{eff}} = q_* \, C_{L,i}, \qquad C_{D,i}^{\text{eff}} = q_* \, C_{D,i}, \qquad C_{Y,i}^{\text{eff}} = q_* \, C_{Y,i}.$$

These accumulate directly into the tail totals $C_{L,\text{tail}}$, $C_{D,\text{tail}}$, $C_{Y,\text{tail}}$.

For moments, the $\boldsymbol{r} \times \boldsymbol{F}$ transfer from each surface's AC to the aircraft CoG is applied in the wind-to-body approximation (at small α body axes $\approx$ wind axes):

$$\begin{aligned} \Delta C_{l,\text{tail}} &\mathrel{+}= q_* \, C_{l,i,AC} + (r_y F_z - r_z F_y)/b_{\text{ref}}, \\ \Delta C_{m,\text{tail}} &\mathrel{+}= q_* \, C_{m,i,AC} + (r_z F_x - r_x F_z)/c_{\text{ref}}, \\ \Delta C_{n,\text{tail}} &\mathrel{+}= q_* \, C_{n,i,AC} + (r_x F_y - r_y F_x)/b_{\text{ref}}, \end{aligned} \tag{22}$$

with $F_x = C_{D,i}^{\text{eff}}$, $F_y = C_{Y,i}^{\text{eff}}$, $F_z = -C_{L,i}^{\text{eff}}$ (sign flip: body-frame z down, lift is up), and $(r_x, r_y, r_z) = \boldsymbol{r}_{AC,i} - \boldsymbol{r}_{CoG}$ in body axes.

4.6 Total whole-aircraft coefficient

Finally, the wing+body contribution is added to the tail totals to give the whole-aircraft coefficients delivered to the 6-DoF solver:

$$\boxed{\begin{aligned} C_{X,\text{tot}} &= C_{X,\text{wb}}(\alpha, \beta) + \sum_i \eta_i \, C_{X,i}(\alpha_h, \beta_v), \qquad X \in \{L, D, Y\}, \\ C_{M,\text{tot}} &= C_{M,\text{wb}}(\alpha, \beta) + \sum_i \Big[\eta_i \, C_{M,i,AC}(\alpha_h, \beta_v) + (\boldsymbol{r}_i \times \boldsymbol{F}_i)/\text{scale}_M\Big], \quad M \in \{l, m, n\}. \end{aligned}} \tag{23}$$

where $\text{scale}_m = c_{\text{ref}}$ and $\text{scale}_{l,n} = b_{\text{ref}}$.

4.7 Dynamic stall blend on wing+body

For the wing+body contribution we further apply a one-state Kirchhoff-style dynamic stall model that tracks an effective lagged angle α_{lag} and a stall intensity $\sigma_{\text{s}} \in [0, 1]$:

$$\dot{\alpha}_{\text{lag}} = \frac{\alpha + k_q \hat{q} - \alpha_{\text{lag}}}{\tau_\alpha}, \tag{24}$$

$$\dot{\sigma}_{\text{s}} = \frac{\sigma_{\text{s,target}}(\alpha_{\text{lag}}) - \sigma_{\text{s}}}{\tau_\sigma^{\uparrow/\downarrow}}, \tag{25}$$

$$\sigma_{\text{s,target}}(\alpha_{\text{lag}}) = \begin{cases} 1 & |\alpha_{\text{lag}}| \geq \alpha_{\text{on}}, \\ 0 & |\alpha_{\text{lag}}| \leq \alpha_{\text{off}}, \\ \text{previous} & \text{otherwise (hysteresis)}, \end{cases} \tag{26}$$

$$C_L^{\text{blend}} = (1 - w)\, C_L^{\text{pre}} + w\, C_L^{\text{post}}, \tag{27}$$

$$w(\sigma_{\text{s}}) = \sigma_{\text{s}}^2 (3 - 2\sigma_{\text{s}}) \quad \text{(smoothstep)}. \tag{28}$$

The pre-stall value C_L^{pre} comes from the assembled v3 lookup; the post-stall value $C_L^{\text{post}} = K \sin(2\alpha_{\text{lag}})$ is a fully separated flat-plate form. The two time constants encode hysteresis: rise (flow separation onset) is fast ($\tau_\sigma^\uparrow \sim 50\,\text{ms}$), reattachment is slow ($\tau_\sigma^\downarrow \sim 500\,\text{ms}$).

The two states α_{lag} and σ_{s} are carried as the last two slots of the aircraft state vector (indices 14 and 15), and their time derivatives (24) and (25) are computed and integrated alongside the position/velocity/attitude by the RK4 driver.

4.8 Worked example — PC21 cruise lookup

To make the above concrete, consider a PC21 trainer in clean config at cruise: $\alpha = 3°$, $\beta = 0°$, $M = 0.2$. From the YAML (PC21):

- $S_{\text{ref}} = 16.3\,\text{m}^2$, $b_{\text{ref}} = 9.11\,\text{m}$, $c_{\text{ref}} = 1.79\,\text{m}$, CoG $= (2.95, 0, 0)$.
- HTail: arm $= (5.65, 0, 0.4)$, $S_h = 3.4\,\text{m}^2$.
- VTail: arm $= (5.45, 0, -0.5)$ (approximately), $S_v = 2.1\,\text{m}^2$.

1. Interference lookups return $\varepsilon(3°) \approx 3.2°$, $\eta_h(3°) \approx 0.93$, $\sigma(0) = 0$, $\eta_v(0) \approx 0.95$.
2. Local tail angles: $\alpha_h = 3° - 3.2° = -0.2°$, $\beta_v = 0°$. The HTail sees almost zero local α at cruise, which is the point of the $-1.5°$ incidence set on the HTail in the ac_data.yaml (downwash cancels it at trim).
3. Wing+body lookup: `wb_CL`(3°, 0°, 0.2) is read from the envelope-extended table, approximately 0.41 for PC21.
4. Tail-HTail lookups at $(-0.2°, 0°)$: $C_{L,h} \approx -0.01$, $C_{D,h} \approx 0.003$, moments at AC ≈ 0.
5. Tail force totals: $C_{L,\text{tail}} = 0.93 \times (-0.01) \approx -0.009$, $C_{D,\text{tail}} = 0.93 \times 0.003 \approx 0.003$.
6. Moment transfer from HTail AC to CoG: $F_z = -(-0.009) = 0.009$, $r_x = 5.65$, so $\Delta C_m \approx (r_z F_x - r_x F_z)/c_{\text{ref}} \approx (0.4 \times 0.003 - 5.65 \times 0.009)/1.79 \approx -0.028$.
7. Total: $C_L \approx 0.41 + (-0.009) = 0.40$, and C_m is dominated by the wing+body term (-0.04 stable) plus the small tail transfer (-0.028). The damping and control terms below augment the static contributions.

5 Flight simulator — full moment sum

The roll, pitch, and yaw moment coefficients delivered to the 6-DoF solver are the sum of six to eight contributions per axis. Each is implemented as its own function in `0.2.2_compute_aerodynamic_moment_coef`

$$
\begin{aligned}
C_{m,\text{total}} = \;& \text{stiffness}(\alpha, \beta, M, \text{cfg}) \qquad [\text{v3: assembler}] \\
+ \;& C_{m_q}(\alpha, M, \text{cfg})\, \hat{q} \\
+ \;& C_{m_{\dot\alpha}}(\alpha, M, \text{cfg})\, \hat{q} \qquad [\dot\alpha \approx q] \\
+ \;& C_{m_{\delta_e}}(\alpha, M, \text{cfg})\, \delta_e \\
+ \;& \underbrace{C_{m_0} + C_{m,\text{trim}}}_{\text{legacy path only}}
\end{aligned}
\tag{29}
$$

In v3 mode the legacy $C_{m_0} + C_{m,\text{trim}}$ term is suppressed because the wb table already has C_{m_0} baked in. This dispatch is done by checking `aircraft_data.use_component_assembly`, which is `true` when the `wb_*` tables are loaded.

Roll. The roll coefficient is assembled similarly with eight terms:

$$
\begin{aligned}
C_{l,\text{total}} = C_{l,\text{stiffness}}(\alpha, \beta, M) \; & [\text{v3: assembler}] \\
& + C_{l_p}\, \hat{p} + C_{l_r}\, \hat{r} + C_{l_{\delta_a}}\, \delta_a + C_{l_{\delta_r}}\, \delta_r.
\end{aligned}
$$

Yaw. Analogously:

$$
\begin{aligned}
C_{n,\text{total}} = C_{n,\text{stiffness}}(\alpha, \beta, M) \; & [\text{v3: assembler}] \\
& + C_{n_r}\, \hat{r} + C_{n_{\dot\beta}}\, (-\hat{r}) + C_{n_p}\, \hat{p} + C_{n_{\delta_r}}\, \delta_r + C_{n_{\delta_a}}\, \delta_a.
\end{aligned}
$$

The $\dot\beta \approx -r$ substitution is applied at the point of consumption in `0.2.2`, so the lag derivative $C_{n_{\dot\beta}}$ appears with an explicit minus sign.

5.1 Stiffness dispatch

Each of the three "stiffness" functions (`pitching_moment_coefficient_due_to_aerodynamic_stiffness`, `rolling_moment_coefficient_due_to_sideslip`, and the yaw counterpart) checks `use_component_assembly` at entry. When true, it calls `assemble_total_force_and_moment_coefficients` and returns the appropriate scalar from the NamedTuple. When false, it falls back to the legacy formula $C_{i_\alpha}\alpha$ or $C_{i_\beta}\beta$.

5.2 Post-stall damping floor

Several aerodynamic databases produce damping derivatives that collapse toward zero (or even reverse sign) in deep stall. That is physically plausible (separated flow doesn't carry lift-based damping) but it leaves the simulator essentially undamped once $\sigma_\text{s} = 1$, letting small rate upsets run away. To prevent that, the 6-DoF solver blends in a conservative damping floor weighted by the dynamic-stall intensity:

$$\Delta C_{l,\text{floor}} = w(\sigma_\text{s})\, C_{l_p,\min}\, \hat{p}, \tag{30}$$

$$\Delta C_{m,\text{floor}} = w(\sigma_\text{s})\, C_{m_q,\min}\, \hat{q}, \tag{31}$$

$$\Delta C_{n,\text{floor}} = w(\sigma_\text{s})\, C_{n_r,\min}\, \hat{r}, \tag{32}$$

with $C_{l_p,\min} = -0.3$, $C_{m_q,\min} = -6$, $C_{n_r,\min} = -0.2$ (hard-coded in `2.2`). Outside stall ($\sigma_\text{s} = 0$) the floor contributes nothing. The floor is additive and never replaces the tabulated value — it only kicks in when the table is weaker than the floor.

6 6-DoF equations of motion

6.1 State vector

The simulator integrates a 15-element state $\boldsymbol{x}$:

$$\boldsymbol{x} = \big(\underbrace{x_{\mathrm{g}}, y_{\mathrm{g}}, z_{\mathrm{g}}}_{\text{pos, global}}, \underbrace{v_{x,\mathrm{g}}, v_{y,\mathrm{g}}, v_{z,\mathrm{g}}}_{\text{vel, global}}, \underbrace{q_1, q_2, q_3, q_0}_{\text{quaternion}}, \underbrace{p, q, r}_{\text{body rates}}, \underbrace{\alpha_{\mathrm{lag}}, \sigma_{\mathrm{s}}}_{\text{dyn. stall}}\big)^\top \tag{33}$$

Linear position and velocity live in the global frame ($x = \mathrm{E}, y = \mathrm{U}, z = \mathrm{N}$) used by the renderer; angular velocity and the aerodynamic/propulsive forces live in the body frame. The quaternion $\boldsymbol{q}$ transforms between them. The last two entries carry the dynamic-stall states (indices 14–15).

6.2 Newton–Euler equations

In body axes, the translational and rotational dynamics are

$$m\,\dot{\boldsymbol{v}}_{\mathrm{b}} = \boldsymbol{F}_{\mathrm{aero}} + \boldsymbol{F}_{\mathrm{prop}} + m\,\mathcal{R}^{-1}(\boldsymbol{q})\,\boldsymbol{g}_{\mathrm{g}} - \boldsymbol{\omega} \times (m\,\boldsymbol{v}_{\mathrm{b}}), \tag{34}$$

$$\mathbf{I}_b\,\dot{\boldsymbol{\omega}} = \boldsymbol{M}_{\mathrm{aero}} + \boldsymbol{M}_{\mathrm{prop}} - \boldsymbol{\omega} \times \mathbf{I}_b\,\boldsymbol{\omega}, \tag{35}$$

where $\mathbf{I}_b$ is the body-frame inertia tensor (computed from the YAML principal moments and principal-axis rotation, see `extract_inertia_tensor` in `0.1`), $\mathcal{R}(\boldsymbol{q})$ is the quaternion rotation matrix, and $\boldsymbol{g}_{\mathrm{g}} = (0, -g, 0)^\top$ is gravity in global axes.

The aerodynamic force and moment are

$$\boldsymbol{F}_{\mathrm{aero}}^{\mathrm{wind}} = \bar{q}_\infty S_{\mathrm{ref}} \begin{pmatrix} -C_D \\ C_Y \\ -C_L \end{pmatrix}, \qquad \boldsymbol{M}_{\mathrm{aero}}^{\mathrm{body}} = \bar{q}_\infty S_{\mathrm{ref}} \begin{pmatrix} b_{\mathrm{ref}}\,C_l \\ c_{\mathrm{ref}}\,C_m \\ b_{\mathrm{ref}}\,C_n \end{pmatrix}, \tag{36}$$

with the force rotated from wind to body via the standard $R_{\mathrm{w}\to\mathrm{b}}(\alpha, \beta)$ matrix. The dynamic pressure is $\bar{q}_\infty = \frac{1}{2}\rho V^2$ with ρ from the ISA-76 atmosphere (`4.2_ISA76.jl`).

6.3 Inertia tensor

The inertia tensor is constructed from the YAML's principal moments and principal-axes pitch angle θ_I (typical aerobatic aircraft have the principal y-axis tilted a few degrees from the body y-axis because the engine and propeller mass distribution is forward of the geometric centroid):

$$\mathbf{I}_b \;=\; R_y(\theta_I)\,\mathrm{diag}(I_{xx}^p, I_{yy}^p, I_{zz}^p)\,R_y(\theta_I)^\top, \tag{37}$$

where R_y is the rotation about body y. The matrix is inverted once at load time and the inverse is cached on `aircraft_data`.

6.4 Quaternion kinematics

The quaternion kinematics use the vectorised product

$$\dot{\boldsymbol{q}} \;=\; \tfrac{1}{2}\,\boldsymbol{q} \otimes \begin{pmatrix} 0 \\ \boldsymbol{\omega}_b \end{pmatrix}, \tag{38}$$

where $\otimes$ denotes Hamilton quaternion multiplication (`1.1_quaternions_and_transformations.jl`). After every RK4 substep a unit-norm renormalisation is applied: $\boldsymbol{q} \leftarrow \boldsymbol{q}/\|\boldsymbol{q}\|$.

6.5 Runge–Kutta 4 integrator

The integrator in `2.1_Runge_Kutta_4_integrator.jl` advances the state from t to $t+\Delta t$ using the classical RK4 scheme

$$\boldsymbol{x}_{t+\Delta t} \;=\; \boldsymbol{x}_t + \tfrac{\Delta t}{6}(\boldsymbol{k}_1 + 2\boldsymbol{k}_2 + 2\boldsymbol{k}_3 + \boldsymbol{k}_4), \tag{39}$$

with $\boldsymbol{k}_1 = f(t, \boldsymbol{x}_t)$, $\boldsymbol{k}_2 = f(t + \Delta t/2,\ \boldsymbol{x}_t + \Delta t\, \boldsymbol{k}_1/2)$, $\boldsymbol{k}_3 = f(t + \Delta t/2,\ \boldsymbol{x}_t + \Delta t\, \boldsymbol{k}_2/2)$, $\boldsymbol{k}_4 = f(t + \Delta t,\ \boldsymbol{x}_t + \Delta t\, \boldsymbol{k}_3)$. The evaluator f at each substep recomputes the full aero stack: flight conditions, anemometry, control attainment, assembler, moment sum, propulsion, gravity. Sub-stepping is enforced with a hard cap (`max_rk4_substep_seconds`, default 0.02 s) so that the integrator degrades gracefully under frame-rate hitches.

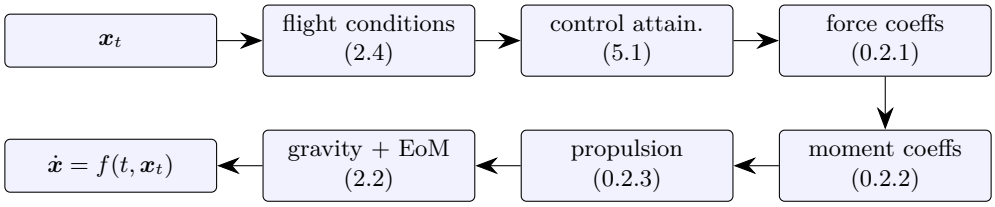


Figure 3: Derivative evaluator inside one RK4 substep. Each arrow represents a function call into the next module, passing the intermediate flight conditions structure as shared state.

7 Anemometry, atmosphere, controls

7.1 ISA-76 atmosphere

`4.2_ISA76.jl` implements the International Standard Atmosphere:

$$T(h) = T_0 - Lh, \quad L = 6.5 \times 10^{-3}\,\mathrm{K\,m^{-1}} \qquad (h < 11\,\mathrm{km}), \tag{40}$$

$$p(h) = p_0\,(1 - Lh/T_0)^{g/(LR)}, \tag{41}$$

$$\rho(h) = p(h)/(RT(h)), \qquad a(h) = \sqrt{\gamma RT(h)}. \tag{42}$$

Above the tropopause the code switches to the isothermal-plus-lapse formulas of the 1976 standard. Dynamic pressure and Mach follow from $\bar{q}_\infty = \frac{1}{2}\rho V^2$ and $M = V/a$.

7.2 Actuator and engine dynamics

Every control demand is passed through a first-order actuator with rate limiting (`5.1_Actuator_and_Engine_Dynam`

$$\dot{\delta} = \mathrm{clamp}\big((\delta_{\mathrm{cmd}} - \delta)/\tau_{\mathrm{act}}, -\dot{\delta}_{\max}, +\dot{\delta}_{\max}\big), \qquad \delta \in [\delta_{\min}, \delta_{\max}]. \tag{43}$$

This is critical for realistic pilot feel — real control surfaces take a finite time to reach the commanded position, and the rate limit is what prevents a step input from producing an infinite initial moment.

Engines follow a similar first-order spool dynamic with separate rise/fall time constants:

$$\dot{T} = \frac{T_{\mathrm{cmd}} - T}{\tau_{\mathrm{spool}}(T_{\mathrm{cmd}}, T)}, \qquad \tau_{\mathrm{spool}} = \begin{cases} \tau_{\mathrm{up}} & T_{\mathrm{cmd}} > T, \\ \tau_{\mathrm{down}} & \text{otherwise.} \end{cases} \tag{44}$$

7.3 Control attainment

The "attained" control demand vector is the input to the aerodynamic coefficient build-up. It consists of the current deflections divided by their maximum values (so each channel is $\in [-1, +1]$) plus the throttle and the active configuration identifier. Legacy aircraft define their control effectiveness in terms of these non-dimensional demand values, so the physical deflection range is re-applied inside `0.2.2` when multiplying by C_{i_δ}.

8 Visualization and inspection

8.1 Aero model viewer (`aero_model_viewer.html`)

The viewer consumes a JSON payload built by `0.4_aero_model_inspector.jl`. For each loaded aircraft it produces a grid of plots:

- Longitudinal coefficients vs. α (C_L, C_D, C_m) with wing+body and per-tail overlay traces.
- Lateral coefficients vs. β (C_Y, C_l, C_n) with the same overlays.
- Four interference plots: $\varepsilon(\alpha)$, $\eta_h(\alpha)$, $\sigma(\beta)$, $\eta_v(\beta)$.
- Dynamic derivatives vs. α ($C_{l_p}, C_{m_q}, C_{n_r}, C_{n_p}, C_{l_r}$ and the lag derivatives $C_{m_{\dot\alpha}}, C_{n_{\dot\beta}}$).
- Control effectiveness vs. α ($C_{l_{\delta_a}}, C_{m_{\delta_e}}, C_{n_{\delta_r}}$).
- Rate-swept damping plots: ΔC vs. body rate at the trim angle (primary damping, cross-damping, and the three secondary rate terms).

Each plot carries a live-state dot driven by the `openflight_telemetry` BroadcastChannel so the user can see the operating point move through the envelope during flight.

8.2 Model Creator results panel

Within the Model Creator page itself, six tabs (CL vs α, drag polar, Cm vs α, lateral, controls, derivatives) give an immediate preview of the generated database. In v3 mode these are augmented with dashed wing+body and per-tail traces, plus four new tabs for $\varepsilon(\alpha), \eta_h(\alpha), \sigma(\beta), \eta_v(\beta)$.

9 Nomenclature

Symbol	Meaning
α, β	Aircraft-frame angle of attack, sideslip
α_h, β_v	Local tail angles after downwash/sidewash correction
ε, σ	Downwash, sidewash (rad, table-stored in deg)
η_h, η_v	Horizontal/vertical tail dynamic-pressure ratio, $q_{\text{tail}}/\bar{q}_\infty$
$S_{\text{ref}}, b_{\text{ref}}, c_{\text{ref}}$	Aircraft reference area, span, mean aerodynamic chord
$\bar{q}_\infty$	Freestream dynamic pressure $\frac{1}{2}\rho V^2$
$C_{L_\alpha}, C_{Y_\beta}, \ldots$	Stability derivatives
C_{i_j}	Dynamic (rate) derivative $\partial C_i/\partial \hat{j}$
$\hat{p}, \hat{q}, \hat{r}$	Non-dim body rates $pb/(2V), qc/(2V), rb/(2V)$
V_H, V_V	Horizontal- and vertical-tail volume coefficients
$\boldsymbol{q}$	Orientation quaternion (scalar part q_0)
$\mathbf{I}_b$	Body-frame inertia tensor ($\text{kg}\,\text{m}^2$)
σ_{s}	Dynamic-stall intensity state, in $[0, 1]$
ℓ_t, ℓ_v	HTP/VTP lever arm (m, AC minus CoG along x)
λ	Taper ratio c_t/c_r
Λ	Quarter-chord sweep angle (rad)

A Program flow

This appendix traces the concrete sequence of calls made by both applications from launch to steady operation.

A.1 Model Creator startup

1. User runs `julia RunModelCreator.jl`. The script activates the `CREATE_AIRCRAFT_MODEL` Julia project and installs any missing packages.
2. A free port starting from 8765 is located and written back into `src/js/analysis-setup.js` so the browser knows where to open the WebSocket.
3. The `AeroModel` module is loaded via `include` of `src/AeroModel/AeroModel.jl`.
4. The browser opens to `create_aircraft_model.html` in the user's default browser (Edge, Chrome, or Firefox in that order).
5. `AeroModel.start_server` blocks on the WebSocket loop. Requests from the browser ("run analysis", "export YAML") call back into `run_analysis` which executes the pipeline of Fig. 2 and streams progress updates.

A.2 Flight simulator startup

1. User runs `julia OpenFLIGHT.jl`. The entry script `includes`, in order: the package setup, the aerodynamic data/code files (`0.2.4` → `0.2.5` → `0.1` → `0.2.1` → `0.2.2` → `0.2.3` → `0.3` → `0.4`), math helpers (`1.1`, `1.2`), the simulation engine (`2.1` through `2.4`), the WebSocket plumbing (`3.0` through `3.3`), and the atmosphere/anemometry/actuator modules (`4.1-4.3`, `5.1`). The order is critical: `0.2.5` must load before `0.1` because `0.1` calls `has_v3_split_tables` at NamedTuple construction time.
2. `0.2.4` parses the YAML and builds the `AeroData` struct (`parse_aero_data`). This includes the v3 block loader that surfaces `wb_*`, `tail_*_*`, and `interference_*` as namespaced keys.
3. `0.1` constructs `aircraft_flight_physics_and_propulsive_data`, a NamedTuple that holds references to the aero database, mass/inertia, reference geometry, engines, stall parameters, and the v3 flag `use_component_assembly`.
4. `start_server` (in `3.1`) opens a non-blocking WebSocket listener.
5. `launch_client(project_dir)` (in `3.0`) opens the browser to the flight cockpit.
6. `wait_for_server()` blocks the main thread. Once the browser connects, the integrator starts, driven by a setTimeout loop on the JavaScript side that sends control demands every 20 ms.

A.3 Per-frame integration flow

Every Δt (typically 20 ms), the integrator in `2.1` performs:

1. `rk4_step!(state, dt)` is called by the WebSocket handler. It subdivides dt into at most `max_rk4_substep_seconds`-sized substeps.
2. For each substep, $\boldsymbol{k}_{1..4}$ are evaluated by `compute_state_derivative` which in turn calls:
   (a) `compute_instantaneous_flight_conditions` (2.4): builds the named-tuple $(\alpha, \beta, V, M, \bar{q}_\infty, \rho, \ldots)$.
   (b) `attain_control_demands` (5.1): applies the actuator first-order lags to the latest commanded demands.
   (c) `compute_lift/drag/sideforce_coefficient` (0.2.1): returns (C_L, C_D, C_Y) including the dynamic-stall blend and (in v3 mode) a call to `assemble_total_force_and_moment_coefficients`.
   (d) The three stiffness functions, three damping functions, three cross-damping functions, three control functions, and the two lag functions in `0.2.2` — summed per axis into roll, pitch, yaw moment coefficients.
   (e) `compute_propulsive_forces` (0.2.3): returns $\boldsymbol{F}_{\text{prop}}$, $\boldsymbol{M}_{\text{prop}}$ per engine.

(f) `compute_6dof_state_derivative` (2.2): assembles $m\dot{\boldsymbol{v}} = \cdots$ and $\mathbf{I}\dot{\boldsymbol{\omega}} = \cdots$, rotates gravity, and returns $\dot{\boldsymbol{x}}$.

3. Weighted sum $\boldsymbol{x}_{t+\Delta t} = \boldsymbol{x}_t + \Delta t/6 \sum w_i \boldsymbol{k}_i$.
4. Quaternion renormalisation.
5. Collision check (`2.3`).
6. Telemetry packet built and broadcast (`3.2`).

B Function reference

Function signatures cited below use the simplified names at which the code exports them; emoji prefixes and Unicode markers used in the actual file layout are omitted for readability.

B.1 Model Creator — AeroModel module

Function (file)	Role
`run_analysis` (`AeroModel.jl`)	Top-level pipeline: parse, run backends, extend envelope, merge, validate.
`parse_aircraft_input` (`input.jl`)	YAML → `AircraftInput`.
`parse_naca_geometry` (`input.jl`)	NACA designator → (t/c, camber, camber position, LE radius).
`wing_planform` (`input.jl`)	Planform geometry (root/tip chord, span, MAC, LE sweep, dihedral).
`estimate_oswald_factor` (`input.jl`)	AR → Oswald $e \in [0.55, 0.95]$.
`classify_role` (`component_split.jl`)	Role string → {`wing_body`, `tail_h`, `tail_v`}.
`surface_aerodynamic_center` (`component_split.jl`)	MAC station → AC body coords (quarter-chord).
`tail_arm_vector` (`component_split.jl`)	$\boldsymbol{r}_{AC} - \boldsymbol{r}_{CoG}$.
`translate_moment_coefficients` (`component_split.jl`)	$r \times F$ with per-axis reference-length scaling.
`run_vlm_backend` (`vlm_backend.jl`)	Full (α, β) sweep; returns static + derivatives + per-role split.
`build_vlm_geometry` (`vlm_backend.jl`)	Surface-by-surface panel mesh construction.
`extract_per_surface_forces` (`vlm_backend.jl`)	Kutta–Joukowski sum grouped by surface index.
`compute_interference_from_vlm!` (`vlm_backend.jl`)	$\varepsilon(\alpha)$, $\sigma(\beta)$, η_h, η_v tables.
`run_javl_backend` (`javl_backend.jl`)	AVL runner; strip-aggregated per-surface CL.
`run_datcom_backend` (`datcom_backend.jl`)	DATCOM grid over (mach, α).
`datcom_calculate_point` (`datcom_backend.jl`)	Single-point DATCOM: all coefficients at (α, M).
`build_datcom_state` (`datcom_backend.jl`)	Input → DATCOM state dictionary.
`extend_to_full_envelope` (`full_envelope.jl`)	$\pm 180°$ extension on (α, β) grid.
`extend_split_blocks` (`full_envelope.jl`)	Build wing_body/tail/interference for the full grid.
`envelope_CL`, `envelope_CD`, `envelope_CY`, `envelope_Cm`, `envelope_Cl`, `envelope_Cn` (`full_envelope.jl`)	Individual phenomenological models.
`compute_aircraft_stall` (`stall_estimation.jl`)	Geometry-based $C_{L,\max}, \alpha_{\text{stall}}, C_{D_0}, C_{D_{90}}$.
`merge_results` (`merge.jl`)	Priority merge of backends → model Dict.
`build_static_coefficients`, `build_dynamic_derivatives`, `build_control_effectiveness`, `build_control_drag_increments`, `build_local_flow`, `build_poststall` (`merge.jl`)	Individual block constructors.
`build_wing_body_block`, `build_tail_block`, `build_interference_block` (`merge.jl`)	v3.0 split-block constructors.
`get_merged_coefficient`, `get_merged_derivative` (`merge.jl`)	Priority lookup across backends.
`write_yaml`, `write_json`, `model_to_yaml_string` (`output.jl`)	Output writers.
`validate_aero_model`, `report_to_dict` (`validation.jl`)	Quality checks.

B.2 Flight simulator

Function (file)	Role
`parse_aero_data` (0.2.4)	YAML → `AeroData` struct.
`has_aero_coefficient` (0.2.4)	Name lookup with alias resolution.
`fetch_value_from_aero_database` (0.2.4)	Tensor multilinear interpolation at given kwargs.
`fetch_required_numeric_constant`, `fetch_optional_constant` (0.2.4)	Scalar constant fetches with alias fallback.
`extract_tail_surface_geometry` (0.2.4)	v3.0 tail metadata reader.
`has_v3_split_tables` (0.2.5)	True iff wb_* tables present.
`assemble_total_force_and_moment_coefficients` (0.2.5)	**Main assembler.** Local-angle + η + $r \times F$.
`compute_wing_force_coefficients_with_dynamic_stall` (0.2.1)	C_L, C_D, C_S with dynamic-stall blend.
`compute_dynamic_stall_state_derivatives` (0.2.1)	Lagged-α and σ_s time derivatives.
`rolling_moment_coefficient_due_to_sideslip`, `_stiffness` siblings (0.2.2)	Dihedral effect / yaw-stiffness / pitch-stiffness (v3 aware).
`rolling/pitching/yawing_moment_coefficient_due_to_aerodynamic_damping` (0.2.2)	Primary damping.
`yawing_moment_coefficient_due_to_roll_rate`, `rolling_moment_coefficient_due_to_yaw_rate` (0.2.2)	Cross-damping.
`pitching_moment_coefficient_due_to_alpha_dot`, `yawing_moment_coefficient_due_to_beta_dot` (0.2.2)	Lag damping ($\dot{\alpha} \approx q$, $\dot{\beta} \approx -r$).
`rolling/pitching/yawing_moment_coefficient_due_to_control_effectiveness` (0.2.2)	Aileron / elevator / rudder effectiveness.
`compute_propulsive_forces` (0.2.3)	Thrust vector and engine moment.
`compute_linear_aerodynamic_forces_and_moments` (0.3)	Legacy derivative legacy path.
`build_aero_inspector_payload` (0.4)	Inspector JSON payload.
`_linear_mode_payload`, `_table_mode_payload` (0.4)	Payload builders for each mode.
`compute_instantaneous_flight_conditions` (2.4)	State + atmosphere → flight-condition NamedTuple.
`rk4_step!` (2.1)	Classical RK4 with sub-stepping.
`compute_state_derivative` (2.1)	$\dot{\boldsymbol{x}} = f(t, \boldsymbol{x})$ assembly.
`compute_6dof_state_derivative` (2.2)	Core Newton–Euler + moment sum.
`handle_collisions!` (2.3)	Ground contact and penalty response.
`quaternion_multiply`, `rotate_vector_body_to_global` (1.1)	Quaternion helpers.
`compute_ISA76_atmosphere` (4.2)	T, p, ρ, a at altitude.
`compute_anemometry` (4.3)	Wind-relative velocity and Mach.
`attain_control_demands` (5.1)	First-order actuator lag + rate limit.
`update_engine_thrust` (5.1)	Spool dynamics.
`start_server`, `launch_client` (3.0, 3.1)	Web server bootstrap.
`broadcast_aircraft_state` (3.2)	Telemetry packet emission.
`record_and_save_flight_data` (3.3)	Optional CSV logger.

B.3 Source file map (at-a-glance)

Simulator file	Responsibility
`0.1_aircraft_aerodynamic_and_propulsive_data.jl`	Loads YAML into the `aircraft_data` named tuple
`0.2.1_compute_aerodynamic_force_coeffs.jl`	C_L, C_D, C_Y with dynamic-stall blend
`0.2.2_compute_aerodynamic_moment_coeffs.jl`	C_l, C_m, C_n full sum (stiffness, damping, control, lag)
`0.2.3_compute_propulsive_forces.jl`	Engine thrust and moment vectors
`0.2.4_get_constants_and_interpolate_coefficients.jl`	Tensor lookup engine; alias resolution; v3 loader
`0.2.5_assemble_from_components.jl`	Schema v3.0 assembler: $\alpha_h, \beta_v, \eta, r \times F$
`0.3_linear_aerodynamic_model.jl`	Scalar-derivative fallback
`0.4_aero_model_inspector.jl`	Viewer payload builder
`1.1_quaternions_and_transformations.jl`	Quaternion math and frame transforms
`2.1_Runge_Kutta_4_integrator.jl`	Time-step integration + derivative evaluator
`2.2_compute_6DOF_equations_of_motion.jl`	Full EoM: forces, moments, gravity, dynamics
`2.3_handle_collisions.jl`	Ground contact and collision response
`2.4_compute_instantaneous_flight_conditions.jl`	$V, \alpha, \beta, M, \rho, \bar{q}_\infty$ from state
`4.2_ISA76.jl`	Standard atmosphere
`4.3_anemometry.jl`	Wind-relative velocities
`5.1_Actuator_and_Engine_Dynamics.jl`	Control-surface and thrust lags

Model Creator file	Responsibility
`input.jl`	`ac_data.yaml` parser; NACA geometry
`component_split.jl`	Role classification; tail AC; $r \times F$ helpers
`vlm_backend.jl`	VortexLattice.jl driver; per-role aggregation
`javl_backend.jl`	Julia-AVL driver; strip integration
`datcom_backend.jl`	Semi-empirical handbook methods
`full_envelope.jl`	$\pm 180°$ extension; analytic tail reconstruction
`merge.jl`	Priority merge; schema-v3 block builders
`output.jl`	YAML writer (Julia side)
`stall_estimation.jl`	$C_{L,\max}, \alpha_{\text{stall}}$ from airfoil/geometry
`validation.jl`	Sanity-check report on the generated model
`server.jl`	WebSocket server for the browser UI

Version history

Date	Schema	Change
2026-04-13	v3.0	Added wing_body / tail / interference blocks; introduced the component assembler (0.2.5); $r \times F$ moment transfer from each tail AC; downwash / sidewash applied to local tail angles; η_h, η_v scaling of tail forces and moments.
2026-04-14	v3.0+	Added $C_{m_{\dot{\alpha}}}$ and $C_{n_{\dot{\beta}}}$ lag-damping derivatives end-to-end (DATCOM → merge → YAML → 0.2.2 → 2.2). Augments short-period and Dutch-roll damping using the $\dot{\alpha} \approx q$, $\dot{\beta} \approx -r$ approximation.